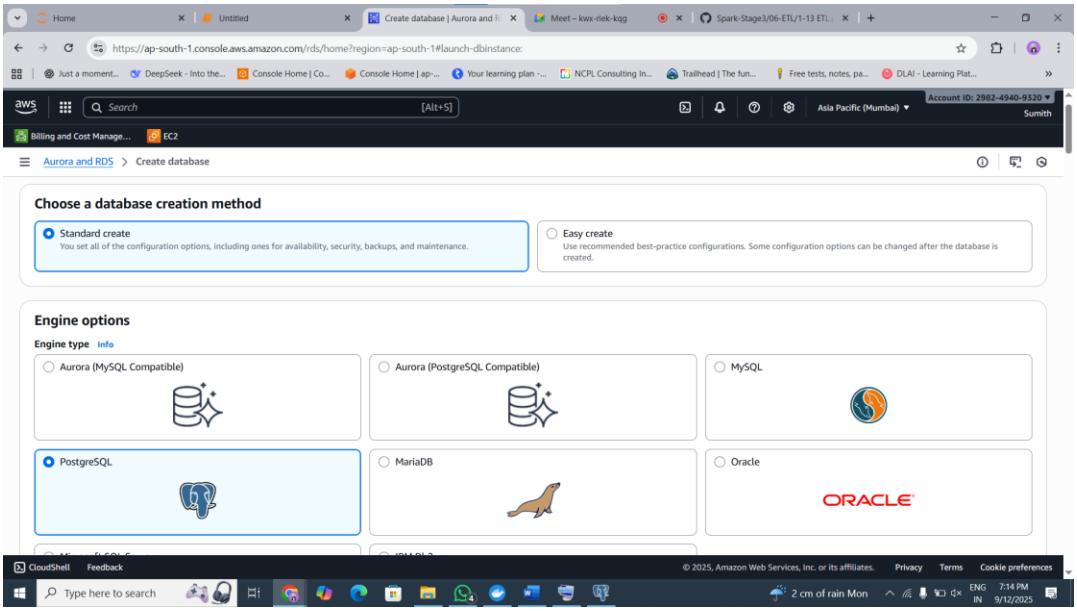
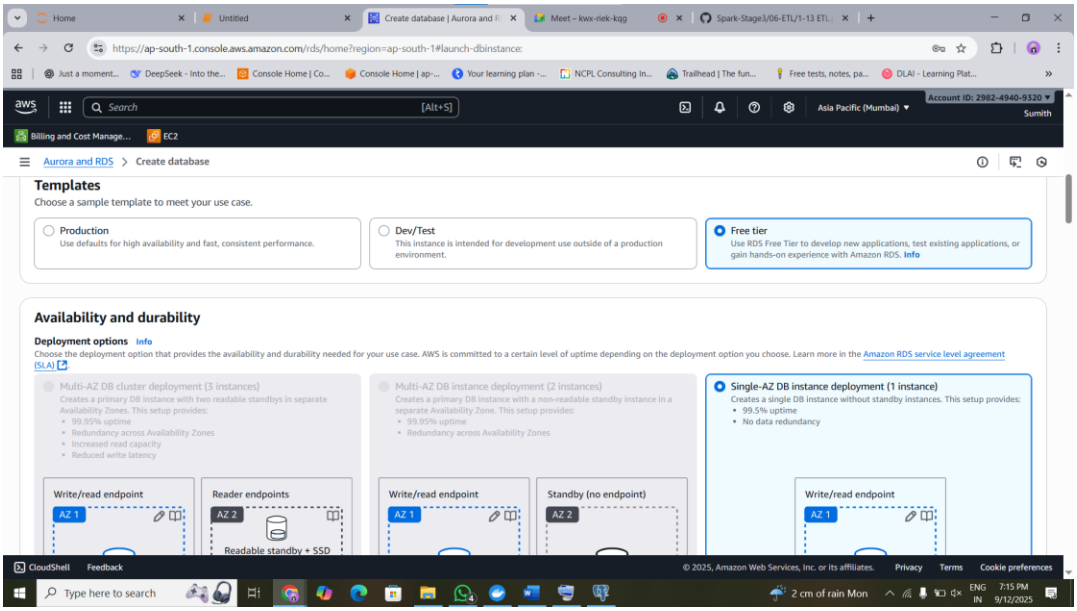
Task: ETL and Database Deployment with AWS RDS + PySpark

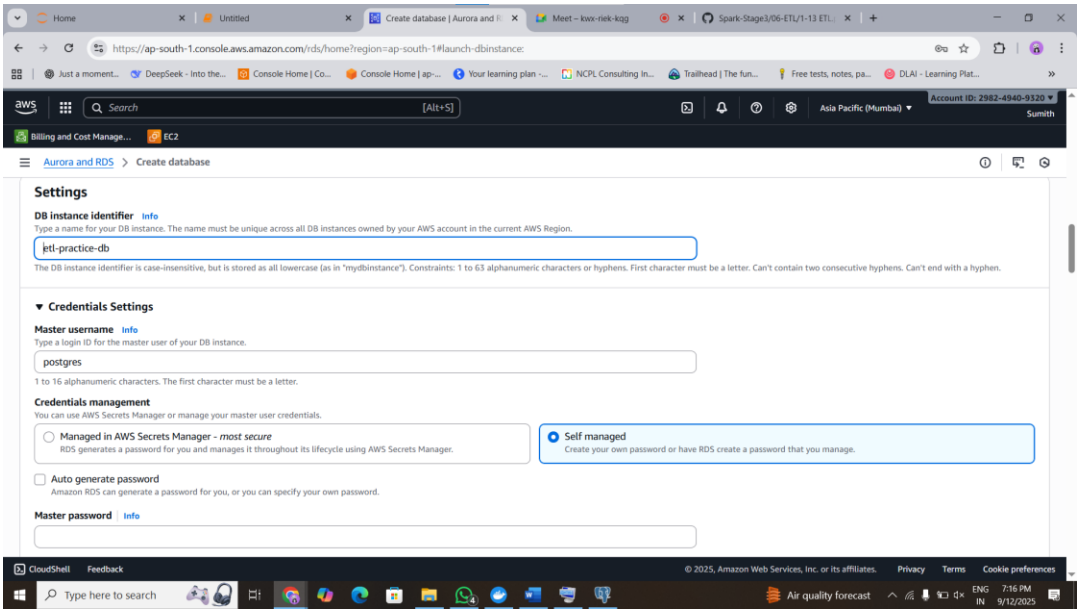
1. Set Up the RDS Database

1. Go to **AWS Console → RDS**.
2. Click **Create database** → choose **Standard Create**.
3. Select **Engine**: PostgreSQL or MySQL (Postgres is commonly used)

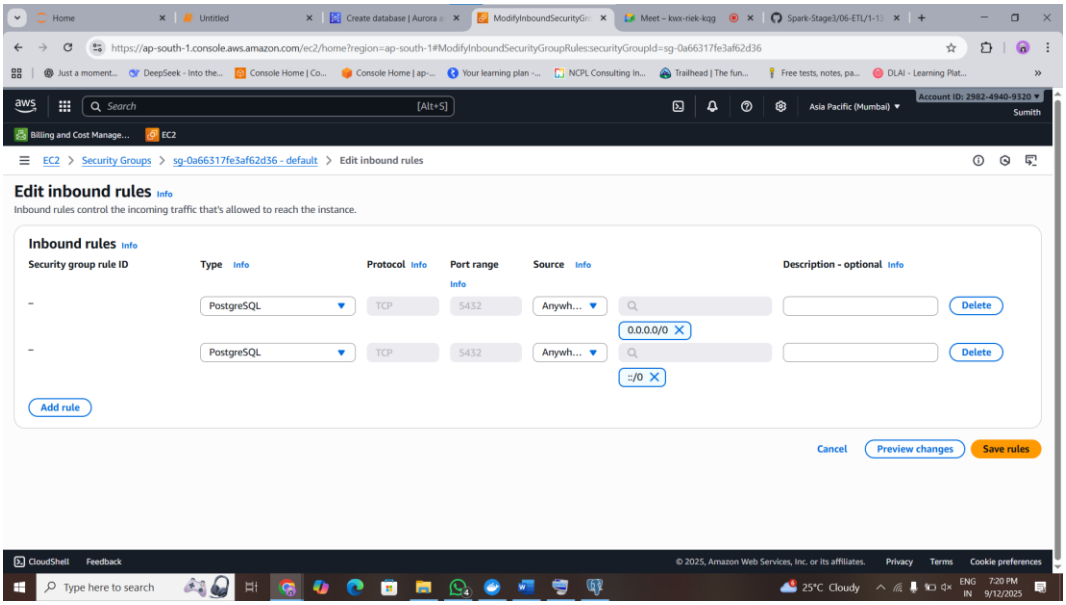




4. Choose a free-tier eligible instance (e.g., db.t3.micro).
5. Set **DB identifier** (e.g., etl-practice-db).

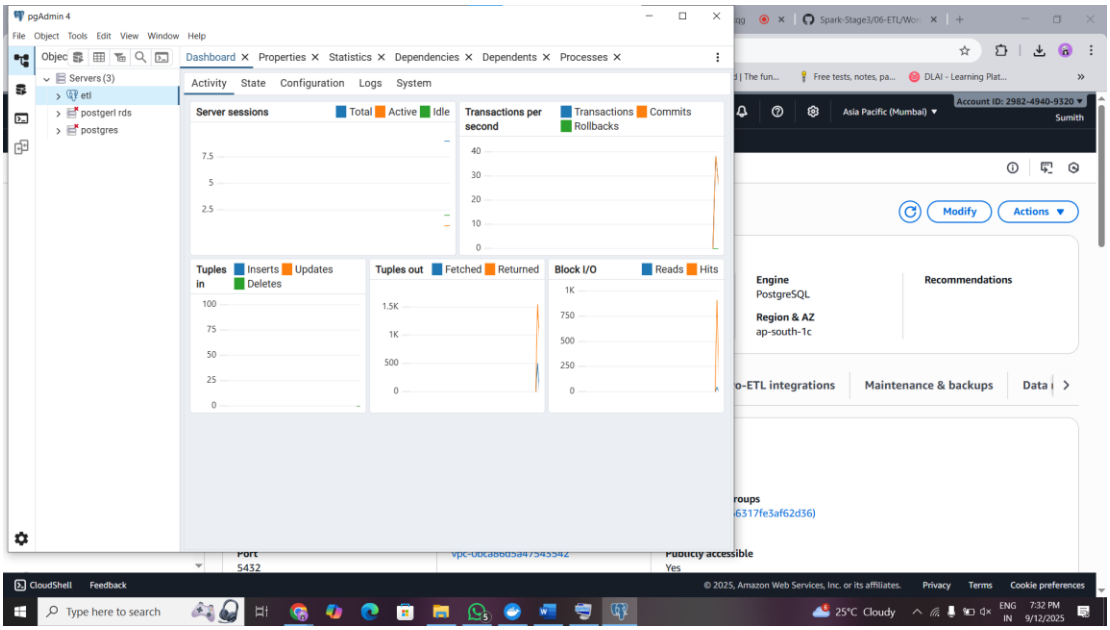


6. Set **username** and **password** → note them down.
7. Configure **VPC and Security Group**:
   - Enable **public access**.
   - Add a security group inbound rule: allow **port 5432 (Postgres)** or **3306 (MySQL)** from your IP.



8. Launch the database and wait until it's **Available**.
9. Copy the **endpoint** (something like etl-practice-db.abc123xyz.us-east-1.rds.amazonaws.com).

   etl-practice-db.ch06icyso7sz.ap-south-1.rds.amazonaws.com



---

docker stop spark-iceberg

docker ps

docker cp "C:\Users\ADMIN\Downloads\postgresql-42.7.7.jar" spark-iceberg:/opt/spark/jars/

[run docker]

docker exec -it spark-iceberg /bin/bash

/opt/spark# ls

ls jars

these code help us to join jdbc.

Select Command Prompt - docker exec -it spark-iceberg /bin/bash

```
C:\Users\WELCOME>docker stop spark-iceberg
spark-iceberg

C:\Users\WELCOME>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED      STATUS                    PORTS
a8af32035f66   minio/mc                       "/bin/sh -c '\nuntil …"  2 days ago   Up 59 minutes
b3e4326c26ad   minio/minio                    "/usr/bin/docker-ent…"   2 days ago   Up 59 minutes             0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001
e165d4dc94ca   apache/iceberg-rest-fixture    "java -jar iceberg-r…"   2 days ago   Up 59 minutes (healthy)   0.0.0.0:8181->8181/tcp, [::]:8181->8181/tcp

C:\Users\WELCOME>docker cp "D:/down/postgresql-42.7.7.jar" spark-iceberg:/opt/spark/jars/
Successfully copied 1.1MB to spark-iceberg:/opt/spark/jars/

C:\Users\WELCOME>docker -it spark-iceberg /bin/bash
unknown shorthand flag: 'i' in -it

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information

C:\Users\WELCOME>docker -it spark-iceberg /bin/bash
unknown shorthand flag: 'i' in -it

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information

C:\Users\WELCOME>docker exec -it spark-iceberg /bin/bash
root@569b95e1b0fb:/opt/spark# /opt/spark# ls
bash: /opt/spark#: No such file or directory
root@569b95e1b0fb:/opt/spark# ls jars
HikariCP-2.5.1.jar                                  kubernetes-httpclient-okhttp-6.7.2.jar
JLargeArrays-1.5.jar                                kubernetes-model-admissionregistration-6.7.2.jar
JTransforms-3.1.jar                                 kubernetes-model-apiextensions-6.7.2.jar
RoaringBitmap-0.9.45.jar                            kubernetes-model-apps-6.7.2.jar
ST4-4.0.4.jar                                       kubernetes-model-autoscaling-6.7.2.jar
activation-1.1.1.jar                                kubernetes-model-batch-6.7.2.jar
aircompressor-0.27.jar                              kubernetes-model-certificates-6.7.2.jar
algebra_2.12-2.0.1.jar                              kubernetes-model-common-6.7.2.jar
annotations-17.0.0.jar                              kubernetes-model-coordination-6.7.2.jar
```

Select Command Prompt - docker exec -it spark-iceberg /bin/bash

```
C:\Users\WELCOME>docker exec -it spark-iceberg /bin/bash
root@569b95e1b0fb:/opt/spark# /opt/spark# ls
bash: /opt/spark#: No such file or directory
root@569b95e1b0fb:/opt/spark# ls jars
HikariCP-2.5.1.jar                                  kubernetes-httpclient-okhttp-6.7.2.jar
JLargeArrays-1.5.jar                                kubernetes-model-admissionregistration-6.7.2.jar
JTransforms-3.1.jar                                 kubernetes-model-apiextensions-6.7.2.jar
RoaringBitmap-0.9.45.jar                            kubernetes-model-apps-6.7.2.jar
ST4-4.0.4.jar                                       kubernetes-model-autoscaling-6.7.2.jar
activation-1.1.1.jar                                kubernetes-model-batch-6.7.2.jar
aircompressor-0.27.jar                              kubernetes-model-certificates-6.7.2.jar
algebra_2.12-2.0.1.jar                              kubernetes-model-common-6.7.2.jar
annotations-17.0.0.jar                              kubernetes-model-coordination-6.7.2.jar
antlr-runtime-3.5.2.jar                             kubernetes-model-core-6.7.2.jar
antlr4-runtime-4.9.3.jar                            kubernetes-model-discovery-6.7.2.jar
aopalliance-repackaged-2.6.1.jar                    kubernetes-model-events-6.7.2.jar
arpack-3.0.3.jar                                    kubernetes-model-extensions-6.7.2.jar
arpack_combined_all-0.1.jar                         kubernetes-model-flowcontrol-6.7.2.jar
arrow-format-12.0.1.jar                             kubernetes-model-gatewayapi-6.7.2.jar
arrow-memory-core-12.0.1.jar                        kubernetes-model-metrics-6.7.2.jar
arrow-memory-netty-12.0.1.jar                       kubernetes-model-networking-6.7.2.jar
arrow-vector-12.0.1.jar                             kubernetes-model-node-6.7.2.jar
audience-annotations-0.5.0.jar                      kubernetes-model-policy-6.7.2.jar
avro-1.11.4.jar                                     kubernetes-model-rbac-6.7.2.jar
avro-ipc-1.11.4.jar                                 kubernetes-model-resource-6.7.2.jar
avro-mapred-1.11.4.jar                              kubernetes-model-scheduling-6.7.2.jar
blas-3.0.3.jar                                      kubernetes-model-storageclass-6.7.2.jar
bonecp-0.8.0.RELEASE.jar                            lapack-3.0.3.jar
breeze-macros_2.12-2.1.0.jar                        leveldbjni-all-1.8.jar
breeze_2.12-2.1.0.jar                               libfb303-0.9.3.jar
cats-kernel_2.12-2.1.1.jar                          libthrift-0.12.0.jar
chill-java-0.10.0.jar                               log4j-1.2-api-2.20.0.jar
chill_2.12-0.10.0.jar                               log4j-api-2.20.0.jar
commons-cli-1.5.0.jar                               log4j-core-2.20.0.jar
commons-codec-1.16.1.jar                            log4j-slf4j2-impl-2.20.0.jar
commons-collections-3.2.2.jar                       logging-interceptor-3.12.12.jar
commons-collections4-4.4.jar                        lz4-java-1.8.0.jar
commons-compiler-3.1.9.jar                          mesos-1.4.3-shaded-protobuf.jar
commons-compress-1.23.0.jar                         metrics-core-4.2.19.jar
commons-crypto-1.1.0.jar                            metrics-graphite-4.2.19.jar
commons-dbcp-1.4.jar                                metrics-jmx-4.2.19.jar
commons-io-2.16.1.jar                               metrics-json-4.2.19.jar
```

```
parquet-encoding-1.13.1.jar
parquet-format-structures-1.13.1.jar
parquet-hadoop-1.13.1.jar
parquet-jackson-1.13.1.jar
pickle-1.3.jar
postgresql-42.7.7.jar
py4j-0.10.9.7.jar
rocksdbjni-8.3.2.jar
scala-collection-compat_2.12-2.7.0.jar
scala-compiler-2.12.18.jar
scala-library-2.12.18.jar
scala-parser-combinators_2.12-2.3.0.jar
```

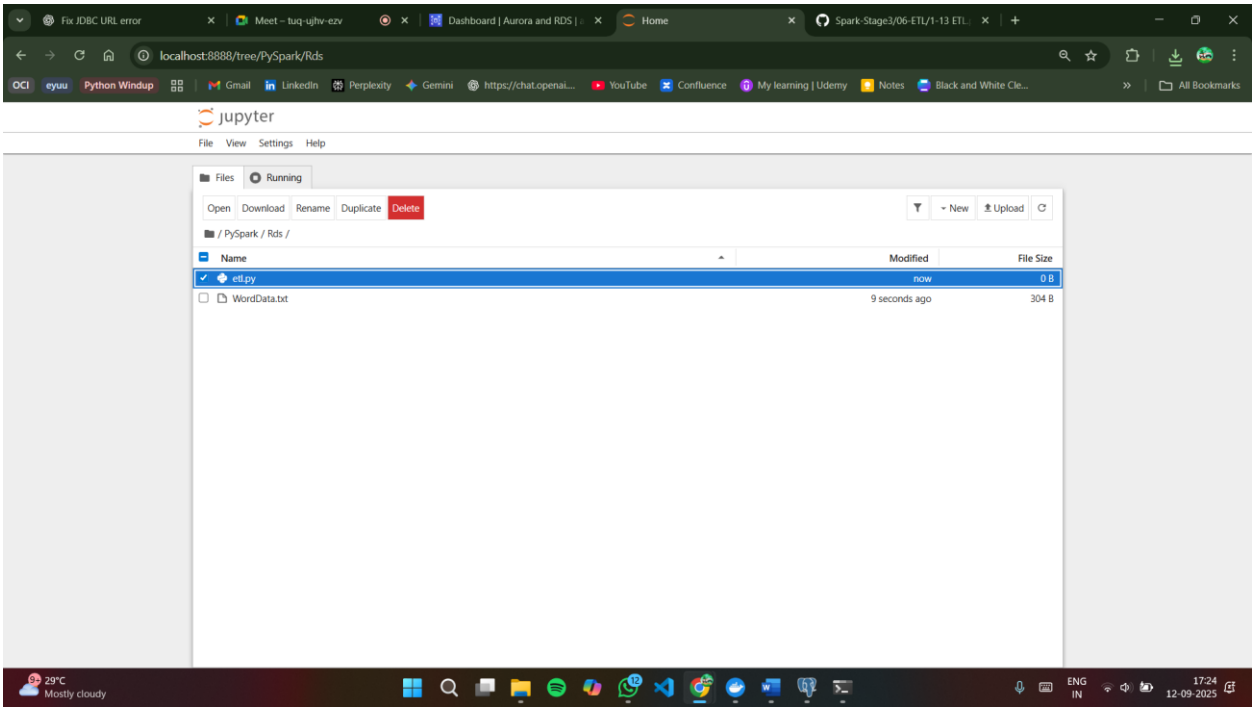
2. Sample ETL Code (PySpark → RDS)

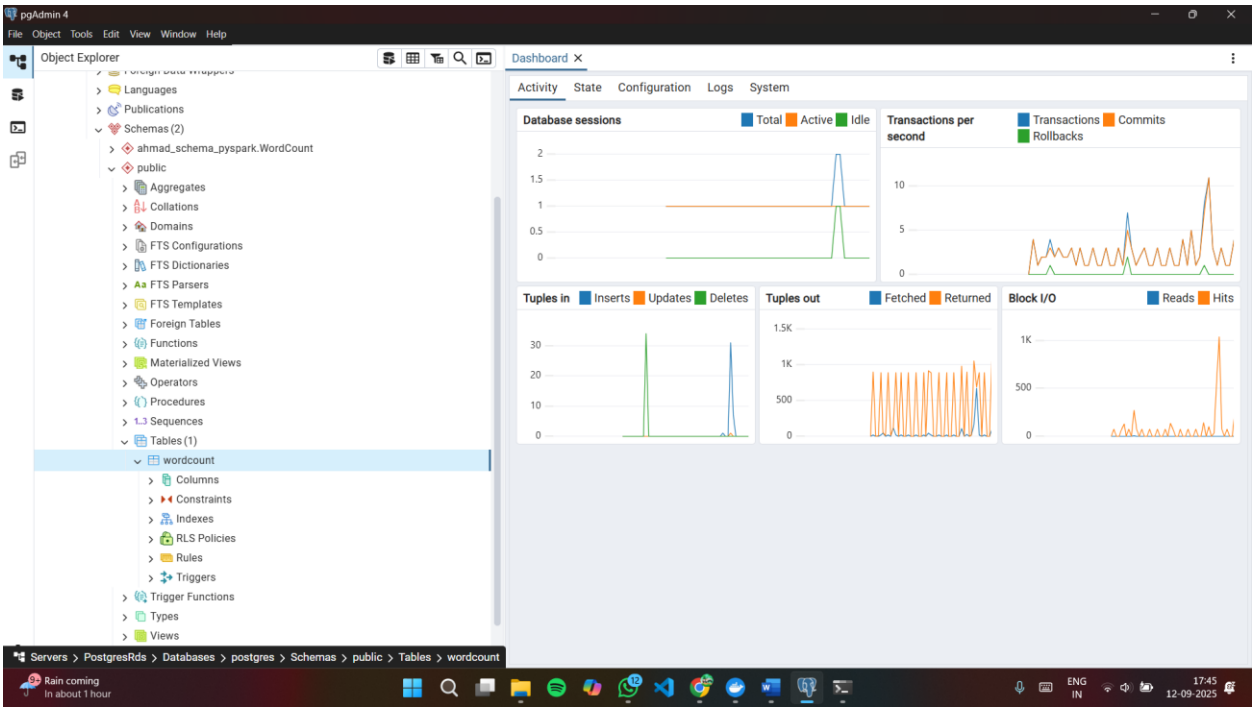
Suppose you have a CSV file with employee data (employees.csv):

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import lit, col, explode
import pyspark.sql.functions as f
# COMMAND ----------
# Extract
spark = SparkSession.builder.appName("ETL Pipeline").getOrCreate()
df = spark.read.text("/FileStore/tables/WordData.txt")
# COMMAND ----------

# Transformation
df2 = df.withColumn("splitedData", f.split("value"," "))
df3 = df2.withColumn("words", explode("splitedData"))
wordsDF = df3.select("words")
wordCount = wordsDF.groupBy("words").count()
# COMMAND ----------
```

```
# Load
driver = "org.postgresql.Driver"
url = "jdbc:postgresql://database-1.c0sanhw4ymut.us-west-2.rds.amazonaws.com/"
table = "ahmad_schema_pyspark.WordCount"
user = "postgres"
password = ""
wordCount.write.format("jdbc").option("driver", driver).option("url",url).option("dbtable", table).option("mode",
"append").option("user",user).option("password", password).save()
```

This will create and load the **employees table** in your RDS.

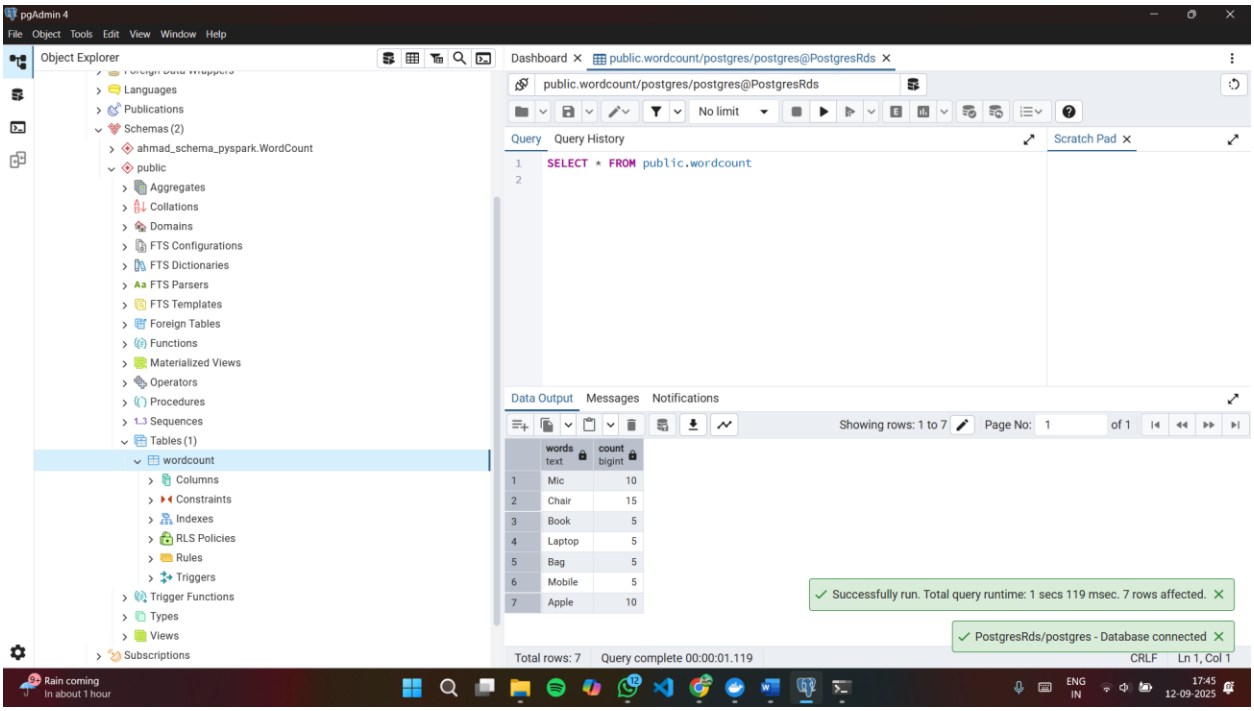




---

5. Validation

- Connect to RDS using a client:
  - PostgreSQL → psql -h <endpoint> -U <username> -d employees_db
  - MySQL → mysql -h <endpoint> -u <username> -p employees_db
- Run:
- SELECT * FROM employees;
- You should see your transformed dataset.

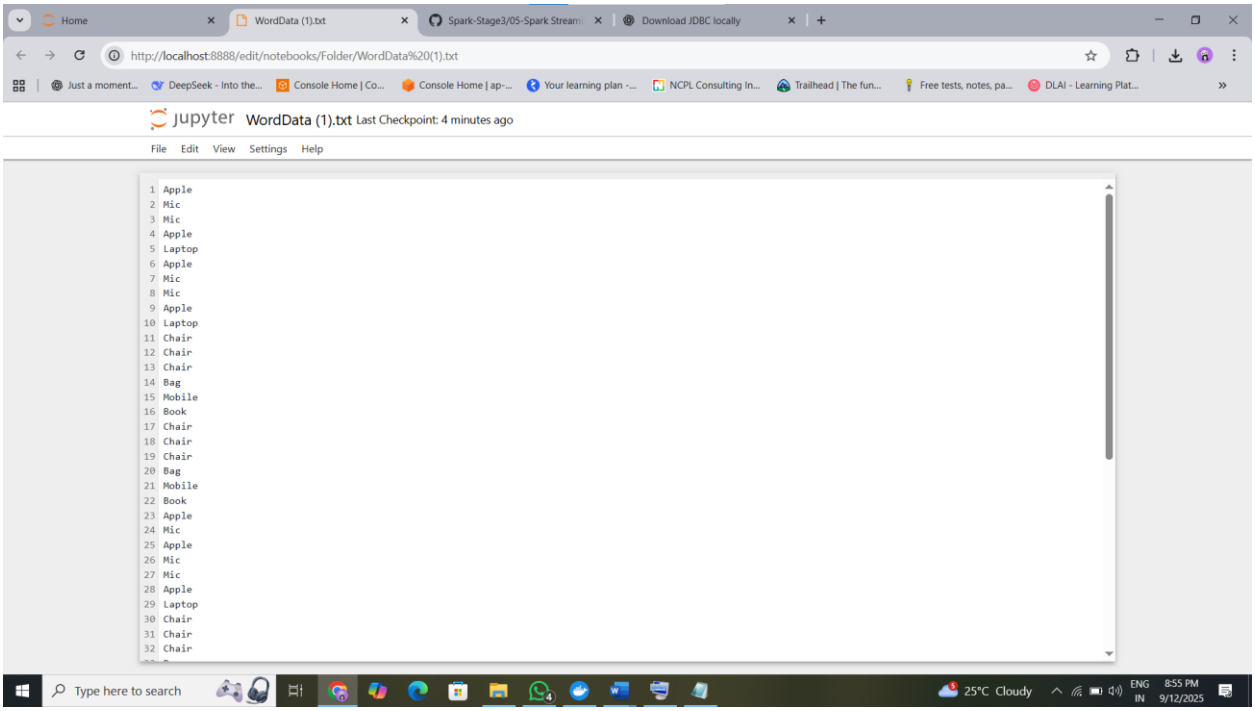


Deliverables

1. **RDS Database deployed** (endpoint, DB name, credentials).
2. **ETL PySpark code** (extract, transform, load).
3. **Uploaded sample dataset** (CSV or other).
4. **Validation query output** showing loaded data.

Step 1: Data file (Python)

This script a new file with Data



Step 2: Spark Streaming Application

This uses textFileStream() to continuously watch the stream_data/ directory.

```
from pyspark.streaming import StreamingContext
from pyspark import SparkConf, SparkContext

conf = SparkConf().setAppName("Streaming")
sc = SparkContext.getOrCreate(conf=conf)

ssc = StreamingContext(sc, 1)

# COMMAND ----------

rdd = ssc.textFileStream("/FileStore/tables/")

# COMMAND ----------

rdd = rdd.map(lambda x: (x,1))
```

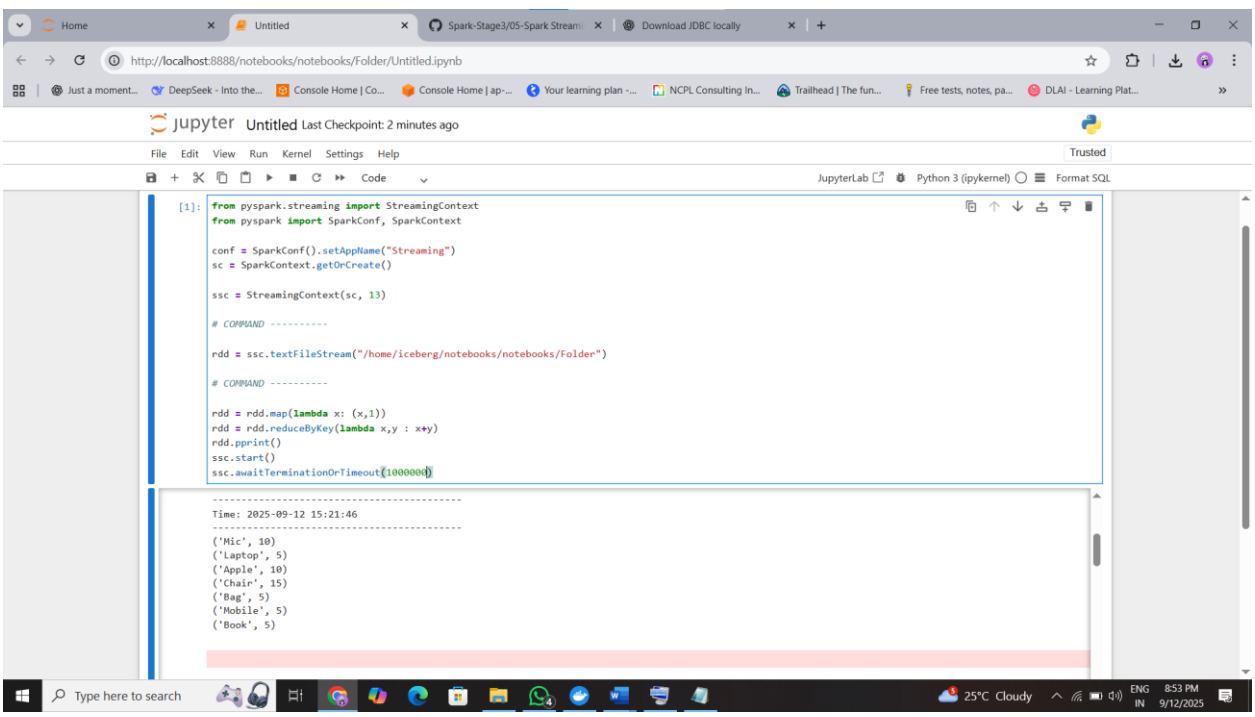
```
rdd = rdd.reduceByKey(lambda x,y : x+y)
rdd.pprint()
ssc.start()
ssc.awaitTerminationOrTimeout(1000000)
```

Step 3: Run

→ This keeps writing new log files every thirteen seconds.

1. In another terminal, run the **Spark Streaming app**:
2. spark-submit spark_streaming_app.py
3. Watch the console output update every 10s with live counts, e.g.:

Time: 2025-09-12 17:32:10



What You Just Practiced

- **File-based streaming** with textFileStream.
- **Near real-time processing** (batch interval = 10s).
- **ETL logic** on streaming data (extract events → transform counts → load to console).

ETL with PySpark + AWS RDS (PostgreSQL)

Step 1. Deploy PostgreSQL on AWS RDS

Follow **Karthick's instructions** (simplified here):

1. Go to **AWS Console → RDS → Create database**.
2. Choose:
   - **Engine:** PostgreSQL
   - **Template:** Free tier
   - **DB instance class:** db.t3.micro
   - **Allocated storage:** 20 GB
   - **DB identifier:** pyspark-etl-db
   - **Master username:** e.g., admin
   - **Password:** choose & save
3. Enable **Public Access** so you can connect.
4. Add a **Security Group inbound rule** to allow **port 5432** from your IP.
5. Launch DB → copy **Endpoint** (looks like: pyspark-etl-db.abc123xyz.us-east-1.rds.amazonaws.com).

(You can observe the screen shot in task 1)

Step 2. Get PostgreSQL JDBC Driver

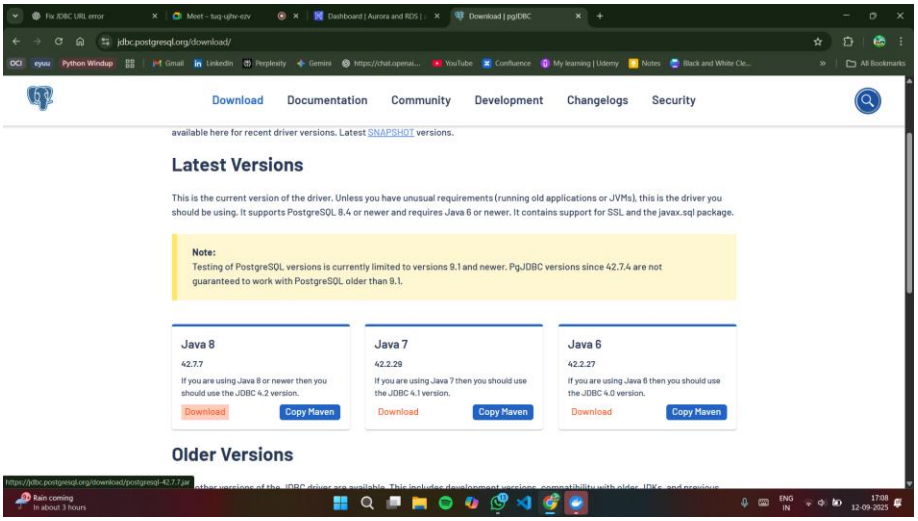
PySpark needs JDBC driver to connect to RDS.

Download:

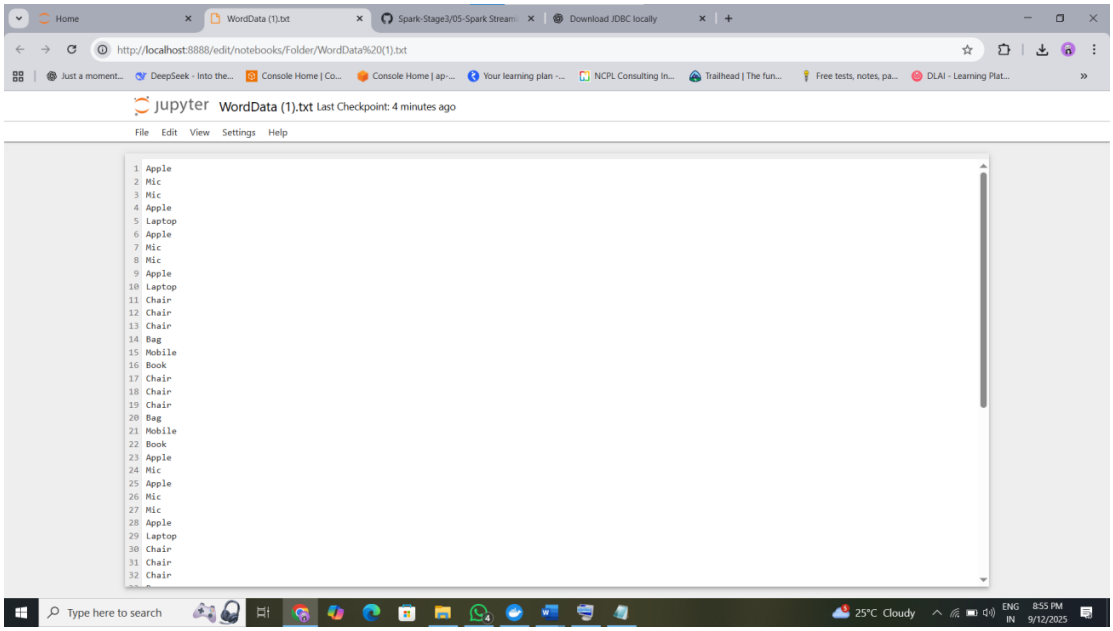
wget https://jdbc.postgresql.org/download/postgresql-42.7.2.jar

When running spark-submit, add:

--jars postgresql-42.7.2.jar

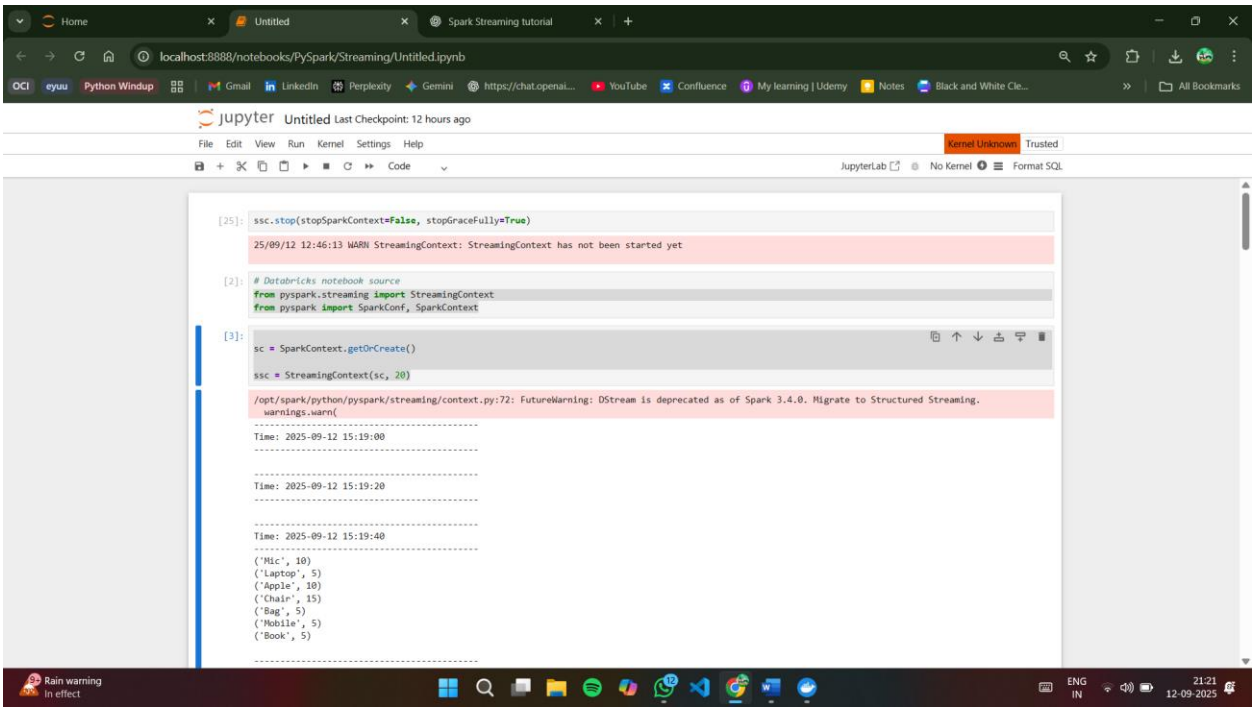


Step 3. Sample Data (txt)



Step 4. PySpark Write ETL (Batch Load to RDS)

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import lit, col, explode

import pyspark.sql.functions as f

# COMMAND ----------

# Extract

spark = SparkSession.builder.appName("ETL Pipeline").getOrCreate()

df = spark.read.text("/FileStore/tables/WordData.txt")

# COMMAND ----------
```



Transformation

df2 = df.withColumn("splitedData", f.split("value"," "))

df3 = df2.withColumn("words", explode("splitedData"))

wordsDF = df3.select("words")

wordCount = wordsDF.groupBy("words").count()

COMMAND ----------

Load

driver = "org.postgresql.Driver"

url = "jdbc:postgresql://database-1.c0sanhw4ymut.us-west-2.rds.amazonaws.com/"

table = "ahmad_schema_pyspark.WordCount"

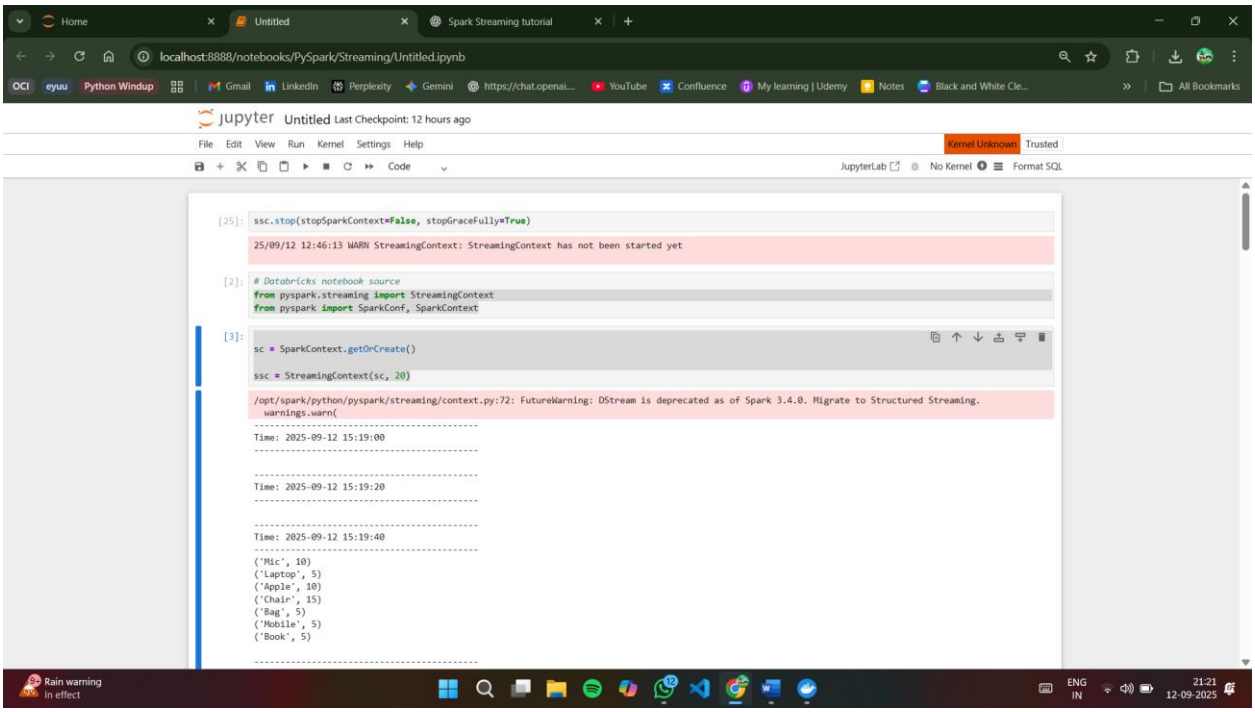
user = "postgres"

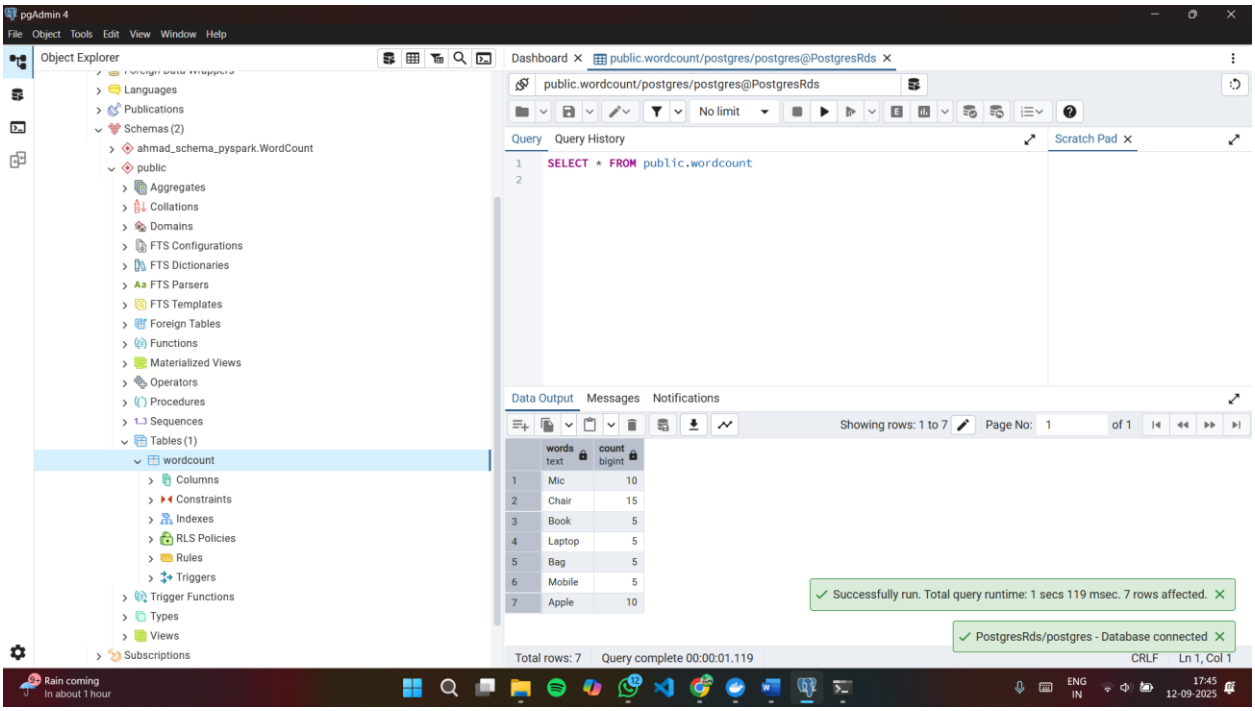
password = ""

wordCount.write.format("jdbc").option("driver", driver).option("url",url).option("dbtable", table).option("mode", "append").option("user",user).option("password", password).save()

Step 5. Read Data Back from RDS

```
# Read back the data
df_from_rds = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "employees") \
    .option("user", db_user) \
    .option("password", db_password) \
    .option("driver", "org.postgresql.Driver") \
    .load()
df_from_rds.show()
```





What You Practiced

- **Extract** → read CSV
- **Transform** → rename, filter nulls
- **Load** → write into PostgreSQL RDS
- **Validate** → read back into Spark

Streaming ETL into RDS with PySpark

We'll use **Structured Streaming** (foreachBatch) since it's the modern API and supports JDBC sinks.

Step 1. Streaming Source (Directory Watcher)

Let's assume you already have your **log generator** writing files into stream_data/.

Step 2. Spark Structured Streaming ETL

This app will read the log files as they appear, aggregate event counts, and **write results into RDS** with each trigger.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import split, col, count

# Initialize Spark
spark = SparkSession.builder \
    .appName("StreamingETLtoRDS") \
    .getOrCreate()

# Step 1: Read streaming data (CSV-like logs)
# Format per line: "timestamp,event"
lines = spark.readStream.text("stream_data")

# Step 2: Extract event column
events = lines.select(split(col("value"), ",").getItem(1).alias("event"))

# Step 3: Transform (count events per batch)
event_counts = events.groupBy("event").agg(count("*").alias("count"))

# RDS connection details
db_endpoint = "pyspark-etl-db.abc123xyz.us-east-1.rds.amazonaws.com"
db_name = "postgres"
db_user = "admin"
db_password = "YourPassword"
db_port = "5432"
```

```
jdbc_url = f"jdbc:postgresql://{db_endpoint}:{db_port}/{db_name}"

# Step 4: Define foreachBatch function
def write_to_rds(batch_df, batch_id):
    batch_df.write \
        .format("jdbc") \
        .option("url", jdbc_url) \
        .option("dbtable", "event_counts_streaming") \
        .option("user", db_user) \
        .option("password", db_password) \
        .option("driver", "org.postgresql.Driver") \
        .mode("append") \
        .save()

# Step 5: Start streaming with foreachBatch
query = event_counts.writeStream \
    .outputMode("complete") \   # output whole count table each trigger
    .foreachBatch(write_to_rds) \
    .trigger(processingTime="10 seconds") \
    .start()

query.awaitTermination()
```

Step 3. What Happens

- Every 10s, Spark:
    1. Reads new log files from stream_data/
    2. Aggregates event counts (CLICK, VIEW, ERROR)
    3. Calls write_to_rds
    4. Persists results in **RDS table event_counts_streaming**

Step 4. Validate in RDS

Connect to your database:

SELECT * FROM event_counts_streaming;

Example output:

event	count
CLICK	12
VIEW	8
ERROR	2

What You Practiced

- **Streaming ETL pipeline**: source → transform → persist.
- **foreachBatch** pattern = write structured streaming micro-batches into DB.
- Persisted results now fuel **dashboards / apps**.

What is Debezium & How It Works

Debezium is an open-source CDC tool that captures row-level changes (inserts, updates, deletes) from databases (like PostgreSQL, MySQL, etc.) by reading their write-ahead log (WAL) or equivalent mechanism. Medium+2Sequin blog+2

With PostgreSQL, Debezium uses **logical replication** (WAL in logical mode) to get a reliable, ordered stream of changes without putting triggers or extra load on the database tables themselves. Medium+1

Debezium runs as a **Kafka Source Connector** inside Kafka Connect. It monitors configured tables/schemas, watches for changes, and publishes change-event messages to Kafka topics. Home+2Medium+2

🏭 **Architecture Diagram: PostgreSQL → Debezium → Kafka → Spark Streaming**

Here's a textual diagram of how components connect; you can sketch similarly visually:

```
┌──────────────┐     ┌──────────────────────┐     ┌──────────────┐     ┌──────────────────────┐
| PostgreSQL   | --->  | Debezium Connector  | --->  |  Kafka / Kafka  | ---> | Spark Structured  |
| (Source DB)  |     | (via WAL / logical) |     |  Topics     |     | Streaming App    |
└──────────────┘     └──────────────────────┘     └──────────────┘     └──────────────────────┘

    |              |                |                |
    | (Write operations)  | (Produces change events) | (Stores & buffers)    | (Consumes & processes)
    |              |                |                |
    |              └── Configuration via Kafka Connect      ──────────┘
    |                  schema/table filters, snapshot      |
    └──────── WAL / Logical Replication settings like `wal_level`,      |
          replication slot, permissions            trigger foreachBatch or        |
                                    windowed aggregations     writes to RDS, dashboards etc.
```

You might also include optional components:

- Schema Registry (for structured events, tracking schema versions)
- Kafka Connect cluster (running Debezium + other connectors)
- Sink consumers (Spark Streaming, other services, dashboards)
- Optionally storage of raw messages for audit, replay

Short Explanation: Benefits vs Simple JDBC Polling / Direct DB Reads

Here are key benefits of a CDC pipeline (PostgreSQL → Debezium → Kafka → Spark) compared to just using JDBC polling or direct querying for changes:

Aspect	JDBC Polling / Direct Querying	CDC with Debezium + Kafka + Streaming
Latency	Often higher — you either poll at intervals, or compare snapshots; can miss timely changes or lag.	Low latency; events are pushed as soon as transactions are committed (via WAL), nearly re time.
Efficiency / Load on DB	Polling frequently can generate heavy load; repeated full scans or timestamp-based queries.	Minimal overhead: WAL is built for durability, logical replication is efficient; no extra polling queries.
Change Types Captured	Usually only new or changed rows if you invent mechanisms; often miss deletes or before-images.	Captures INSERT / UPDATE / DELETE; includes both “before” and “after” state (useful for aud downstream logic).
Ordering & Exactly Once Reliability	Harder to guarantee ordering; possible to miss things or duplicate if polling/persisting snapshots.	Kafka ensures ordered streams per partition (if set up), durable storage; Debezium + Kafka Connect handles offset/state so in failures you can resume.
Scaling / Flexibility	Polling many tables, many sources becomes harder to manage; scaling is manual.	Can scale connectors, topics, consumers separately; add new consumers (Spark, dashboards, other services) without changing source DB.
Replay / Audit / History	Limited: snapshots might lose history; deletes might be hard to track.	Full history of events stored in Kafka; can replay from a point, temporal debugging or reconstruct state.